DAY 28

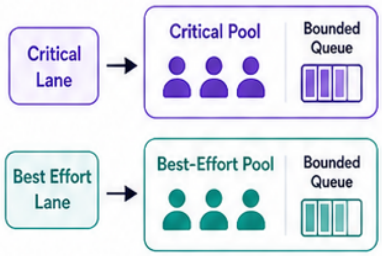
Day 28: Bulkhead Isolation, Worker Pools & Blast Radius Control

Containing failures so one overloaded lane does not sink the entire service.

1 Why Bulkheads Matter

- Shared pools create coupling between unrelated workloads
- Slow dependencies can saturate common resources
- Bulkheads isolate critical and best-effort traffic

2 Isolation Model



- Dedicated worker pools per lane
- Independent queue limits
- Backlog stays local to the affected lane

3 Degradation Behavior

- Non-critical lane can throttle or shed
- Critical lane keeps reserved capacity
- Partial functionality continues during incidents

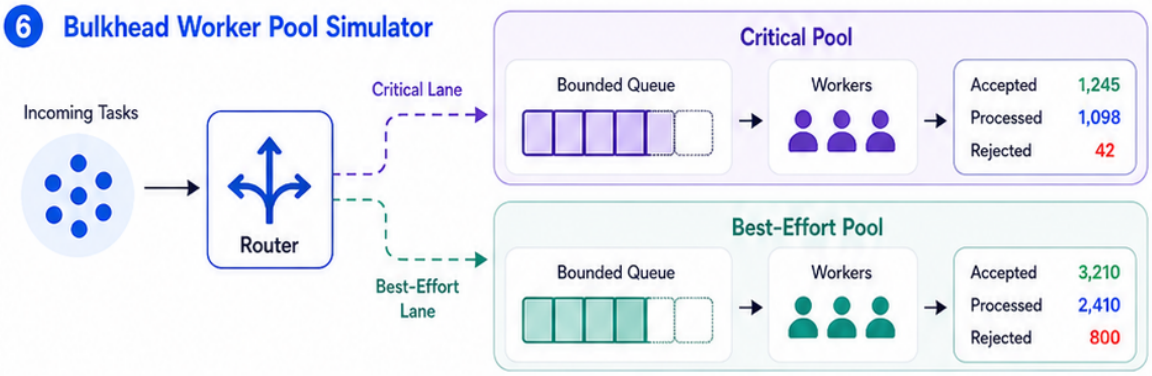
4 Operational Signals

- Per-lane queue depth
- Rejection count
- Completion latency
- Pool-aware alerting

5 Trade-Offs

- Better reliability, slightly lower average utilization
- Over-provisioning wastes capacity
- Under-provisioning sheds too aggressively

6 Bulkhead Worker Pool Simulator



Accepted: Task admitted to queue | Processed: Task completed by worker | Rejected: Task rejected due to limit

Bulkheads reduce blast radius by isolating failures, preserving critical paths, and enabling graceful degradation.

Day 28: Bulkhead Isolation, Worker Pools, and Blast Radius Control

Day 28 focuses on isolating workloads so one failing dependency or noisy traffic class cannot consume all system resources.

It covers:

- bulkhead isolation fundamentals
- dedicated worker pool strategy
- per-pool queue limits
- overload containment behavior
- graceful degradation patterns
- operational trade-offs and observability

In short, Day 28 is about containing failures to one compartment instead of letting them sink the entire service.

Core ideas

Why bulkheads matter

- Shared thread pools and queues create coupling between unrelated workloads.
- A slow downstream call can saturate common resources and block critical paths.
- Bulkheads isolate resources by traffic class or dependency.

Isolation model

- Assign dedicated pool/queue to each workload lane (for example critical vs best_effort).
- Capacity limits are independent per lane.
- Failures and backlog stay local to the overloaded lane.

Degradation behavior

- Non-critical lane can throttle/shed while critical lane remains healthy.
- Critical lane gets reserved capacity and lower tail-latency risk.
- Service can continue partial functionality during incidents.

Operational signals

- Per-lane queue depth, rejection count, and completion latency are key.
- Saturation in one lane should not imply global outage.
- Alerting should be pool-aware to avoid noisy incident triage.

Trade-offs

- Isolation increases reliability but may reduce average utilization.
- Over-provisioned pools waste capacity; under-provisioned pools shed too aggressively.
- Pool sizing should follow workload criticality and SLO targets.

Day-28 sample: Bulkhead Worker Pool Simulator

This repository includes a small bulkhead simulation in both Python and Java.

Functional requirements

- Route tasks to dedicated pools by lane (critical, best_effort)
- Enforce independent queue limits per lane
- Process tasks at fixed throughput per lane
- Reject overflow tasks per lane without affecting the other lane
- Report per-lane accepted/processed/rejected metrics

High-level components

- Task: unit of work tagged with lane
- BulkheadPool: lane-specific bounded queue and worker throughput
- BulkheadRouter: dispatches tasks to the correct pool
- BulkheadEngine: tick-based simulation and metrics aggregation

Data flow

1. Incoming task arrives with lane tag
2. Router forwards task to matching pool
3. Pool accepts or rejects based on lane queue capacity
4. Worker drains fixed number of tasks from each lane per tick
5. Engine reports isolated lane-level metrics

Project structure

```
Day-28/
  README.md
  bulkhead-sample/
    python/
      bulkhead_isolation.py
      demo.py
    java/
      BulkheadSystem.java
      Main.java
```

Run the sample

Python

```
cd bulkhead-sample\python
```

```
python demo.py
```

Java

```
cd bulkhead-sample\java
javac Main.java BulkheadSystem.java
java Main
```

Interview takeaways

- Bulkheads reduce blast radius during localized failures
- Resource isolation should align with business criticality
- Lane-level metrics are required to tune capacity safely
- Graceful degradation is better than full-service collapse
- Bulkheads pair well with retries, backpressure, and circuit breakers

Next improvements

- Add dynamic pool sizing by queue pressure
- Add lane priorities and admission control hints
- Add timeout/cancellation handling per lane
- Add dependency-specific bulkheads (DB, cache, third-party API)
- Add SLO-driven autoscaling based on pool saturation

Code Appendix

Python and Java source used in Day 28 sample.

bulkhead-sample\python\bulkhead_isolation.py

```
from __future__ import annotations

from collections import deque
from dataclasses import dataclass


@dataclass(frozen=True)
class Task:
    task_id: str
    lane: str


class BulkheadPool:
    def __init__(self, lane: str, queue_capacity: int, workers_per_tick: int) -> None:
        self.lane = lane
        self._capacity = queue_capacity
        self._workers_per_tick = workers_per_tick
        self._queue: deque[Task] = deque()

        self.accepted = 0
        self.rejected = 0
        self.processed = 0

    def submit(self, task: Task) -> bool:
        if len(self._queue) >= self._capacity:
            self.rejected += 1
            return False
        self._queue.append(task)
        self.accepted += 1
        return True

    def process_tick(self) -> list[str]:
        history: list[str] = []
        for _ in range(self._workers_per_tick):
            if not self._queue:
                break
            task = self._queue.popleft()
            self.processed += 1
            history.append(f"processed {task.task_id} lane={self.lane}")
        return history

    def queue_depth(self) -> int:
        return len(self._queue)


class BulkheadRouter:
    def __init__(self, pools: dict[str, BulkheadPool]) -> None:
        self._pools = pools

    def route(self, task: Task) -> bool:
        pool = self._pools.get(task.lane)
        if pool is None:
            return False
        return pool.submit(task)


class BulkheadEngine:
    def __init__(self, pools: dict[str, BulkheadPool]) -> None:
        self._pools = pools
        self._router = BulkheadRouter(pools)

    def submit_batch(self, tasks: list[Task]) -> list[str]:
        history: list[str] = []
        for task in tasks:
```

```
            ok = self._router.route(task)
            history.append(("accepted" if ok else "rejected") + f" {task.task_id} lane={task.lane}")
        return history

    def tick(self) -> list[str]:
        history: list[str] = []
        for lane in sorted(self._pools.keys()):
            history.extend(self._pools[lane].process_tick())
        return history

    def metrics(self) -> dict[str, dict[str, int]]:
        result: dict[str, dict[str, int]] = {}
        for lane, pool in self._pools.items():
            result[lane] = {
                "accepted": pool.accepted,
                "rejected": pool.rejected,
                "processed": pool.processed,
                "queue_depth": pool.queue_depth(),
            }
        return result
```

bulkhead-sample\python\demo.py

```
from bulkhead_isolation import BulkheadEngine, BulkheadPool, Task


def main() -> None:
    pools = {
        "critical": BulkheadPool("critical", queue_capacity=5, workers_per_tick=2),
        "best_effort": BulkheadPool("best_effort", queue_capacity=3, workers_per_tick=1),
    }
    engine = BulkheadEngine(pools)

    batch_1 = [
        Task("c-1", "critical"),
        Task("c-2", "critical"),
        Task("b-1", "best_effort"),
        Task("b-2", "best_effort"),
        Task("b-3", "best_effort"),
        Task("b-4", "best_effort"),
    ]

    batch_2 = [
        Task("c-3", "critical"),
        Task("c-4", "critical"),
        Task("c-5", "critical"),
        Task("c-6", "critical"),
        Task("b-5", "best_effort"),
    ]

    print("submit batch 1")
    for line in engine.submit_batch(batch_1):
        print(line)

    print("\ntick 1")
    for line in engine.tick():
        print(line)

    print("\nsubmit batch 2")
    for line in engine.submit_batch(batch_2):
        print(line)

    print("\ntick 2")
    for line in engine.tick():
        print(line)

    print("\nfinal metrics")
    metrics = engine.metrics()
    for lane in sorted(metrics.keys()):
        print(lane, metrics[lane])
```

```
if __name__ == "__main__":
    main()
```

bulkhead-sample\java\BulkheadSystem.java

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class BulkheadSystem {
    public static final class Task {
        private final String taskId;
        private final String lane;

        public Task(String taskId, String lane) {
            this.taskId = taskId;
            this.lane = lane;
        }

        public String getTaskId() {
            return taskId;
        }

        public String getLane() {
            return lane;
        }
    }

    public static final class BulkheadPool {
        private final String lane;
        private final int queueCapacity;
        private final int workersPerTick;
        private final Deque<Task> queue = new ArrayDeque<>();

        private int accepted = 0;
        private int rejected = 0;
        private int processed = 0;

        public BulkheadPool(String lane, int queueCapacity, int workersPerTick) {
            this.lane = lane;
            this.queueCapacity = queueCapacity;
            this.workersPerTick = workersPerTick;
        }

        public boolean submit(Task task) {
            if (queue.size() >= queueCapacity) {
                rejected += 1;
                return false;
            }
            queue.addLast(task);
            accepted += 1;
            return true;
        }

        public List<String> processTick() {
            List<String> history = new ArrayList<>();
            for (int i = 0; i < workersPerTick; i++) {
                if (queue.isEmpty()) {
                    break;
                }
                Task task = queue.removeFirst();
                processed += 1;
                history.add("processed " + task.getTaskId() + " lane=" + lane);
            }
            return history;
        }

        public int getAccepted() {
```

```
        return accepted;
    }

    public int getRejected() {
        return rejected;
    }

    public int getProcessed() {
        return processed;
    }

    public int queueDepth() {
        return queue.size();
    }
}

public static final class BulkheadRouter {
    private final Map<String, BulkheadPool> pools;

    public BulkheadRouter(Map<String, BulkheadPool> pools) {
        this.pools = pools;
    }

    public boolean route(Task task) {
        BulkheadPool pool = pools.get(task.getLane());
        if (pool == null) {
            return false;
        }
        return pool.submit(task);
    }
}

public static final class BulkheadEngine {
    private final Map<String, BulkheadPool> pools;
    private final BulkheadRouter router;

    public BulkheadEngine(Map<String, BulkheadPool> pools) {
        this.pools = pools;
        this.router = new BulkheadRouter(pools);
    }

    public List<String> submitBatch(List<Task> tasks) {
        List<String> history = new ArrayList<>();
        for (Task task : tasks) {
            boolean ok = router.route(task);
            history.add((ok ? "accepted " : "rejected ") + task.getTaskId() + " lane=" + task.getLane()
        }
        return history;
    }

    public List<String> tick() {
        List<String> history = new ArrayList<>();
        List<String> lanes = new ArrayList<>(pools.keySet());
        lanes.sort(String::compareTo);
        for (String lane : lanes) {
            history.addAll(pools.get(lane).processTick());
        }
        return history;
    }

    public Map<String, Map<String, Integer>> metrics() {
        Map<String, Map<String, Integer>> result = new HashMap<>();
        for (Map.Entry<String, BulkheadPool> entry : pools.entrySet()) {
            BulkheadPool p = entry.getValue();
            Map<String, Integer> lane = new HashMap<>();
            lane.put("accepted", p.getAccepted());
            lane.put("rejected", p.getRejected());
            lane.put("processed", p.getProcessed());
            lane.put("queue_depth", p.queueDepth());
            result.put(entry.getKey(), lane);
        }
```

```
            return result;
        }
    }
}
```

bulkhead-sample\java\Main.java

```
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Map<String, BulkheadSystem.BulkheadPool> pools = Map.of(
            "critical", new BulkheadSystem.BulkheadPool("critical", 5, 2),
            "best_effort", new BulkheadSystem.BulkheadPool("best_effort", 3, 1)
        );
        BulkheadSystem.BulkheadEngine engine = new BulkheadSystem.BulkheadEngine(pools);

        List<BulkheadSystem.Task> batch1 = List.of(
            new BulkheadSystem.Task("c-1", "critical"),
            new BulkheadSystem.Task("c-2", "critical"),
            new BulkheadSystem.Task("b-1", "best_effort"),
            new BulkheadSystem.Task("b-2", "best_effort"),
            new BulkheadSystem.Task("b-3", "best_effort"),
            new BulkheadSystem.Task("b-4", "best_effort")
        );

        List<BulkheadSystem.Task> batch2 = List.of(
            new BulkheadSystem.Task("c-3", "critical"),
            new BulkheadSystem.Task("c-4", "critical"),
            new BulkheadSystem.Task("c-5", "critical"),
            new BulkheadSystem.Task("c-6", "critical"),
            new BulkheadSystem.Task("b-5", "best_effort")
        );

        System.out.println("submit batch 1");
        for (String line : engine.submitBatch(batch1)) {
            System.out.println(line);
        }

        System.out.println("\ntick 1");
        for (String line : engine.tick()) {
            System.out.println(line);
        }

        System.out.println("\nsubmit batch 2");
        for (String line : engine.submitBatch(batch2)) {
            System.out.println(line);
        }

        System.out.println("\ntick 2");
        for (String line : engine.tick()) {
            System.out.println(line);
        }

        System.out.println("\nfinal metrics");
        Map<String, Map<String, Integer>> metrics = engine.metrics();
        for (String lane : List.of("critical", "best_effort")) {
            System.out.println(lane + " " + metrics.get(lane));
        }
    }
}
```